

PL/SQL Training

SESSION 5: TRANSACTION MANAGEMENT & WORKING WITH COMPOSITE DATA TYPES

DAVA.X ACADEMY SESSIONS

Svetlana Sura & Iulian Aurel Cozma

10–20 JUNE 2025

SESSION 5

Writing Executable Statements

- Isolation Levels in Oracle
- Use of COMMIT, ROLLBACK, SAVEPOINT
- Transaction lifecycle and control flow
- Behavior of implicit vs. explicit commits
- Handling transactions inside exception blocks

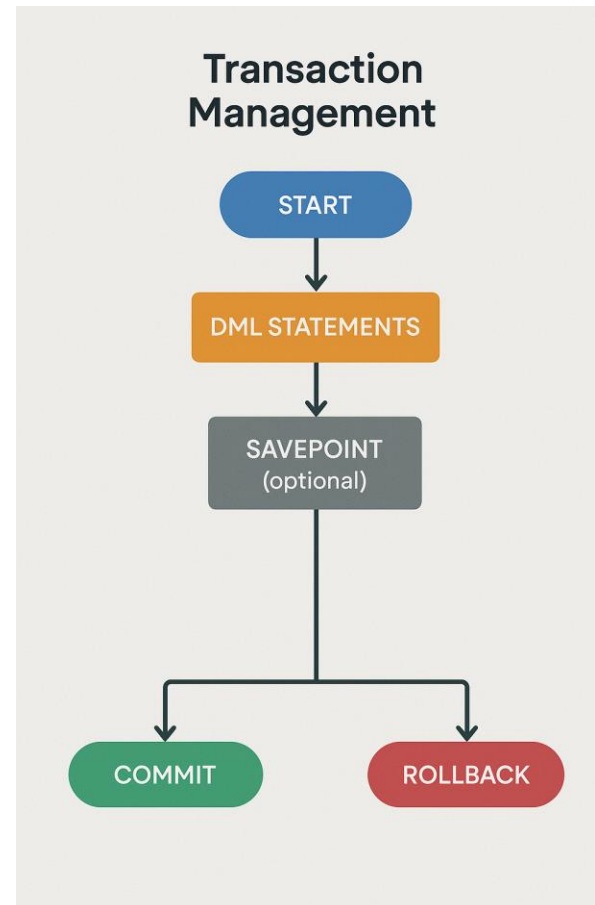


What Is a Transaction

In Oracle, a transaction is a sequence of SQL operations that are executed as a single logical unit of work. A transaction begins with the first DML (Data Manipulation Language) statement (e.g., INSERT, UPDATE, DELETE) and ends with either: **COMMIT** or **ROLLBACK**.

Transaction Flow

1. Start – Implicit when DML begins.
2. DML statements – Modify data.
3. SAVEPOINT (optional) – Mark rollback targets.
4. COMMIT/ROLLBACK – End transaction.



What Is a Transaction



PL/SQL provides the following statements for transaction management:

COMMIT – Saves all outstanding changes since the last **COMMIT** or **ROLLBACK**, and releases all locks.

ROLLBACK – Reverses the effects of all outstanding changes since the last **COMMIT** or **ROLLBACK**, and releases all locks.

ROLLBACK TO SAVEPOINT – Reverses the effects of all changes made since the specified savepoint was established, and releases locks that were established within that range of the code.

SAVEPOINT – Establishes a savepoint, which then allows you to perform partial **ROLLBACKS**.

SET TRANSACTION – Allows you to begin a read-only or read-write session, establish an isolation level, or assign the current transaction to a specified rollback segment.

LOCK TABLE – Allows you to lock an entire database table in the specified mode. This overrides the default row level locking usually applied to a table.

Key Commands in Oracle syntaxes

Statements syntaxes:

COMMIT [**WORK**] [**COMMENT** *text*];

ROLLBACK [**WORK**] [**TO** [**SAVEPOINT**] *savepoint_name*];

SAVEPOINT *savepoint_name*;

LOCK TABLE *table_reference_list* **IN** *lock_mode* **MODE** [**NOWAIT**];

lock_mode

- **ROW SHARE** (RS)
- **ROW EXCLUSIVE** (RX)
- **SHARE** (S)
- **SHARE ROW EXCLUSIVE** (SRX)
- **EXCLUSIVE** (X)

If you specify the **NOWAIT** keyword, the database does not wait for the lock if the table has already been locked by another user, and instead reports an error.

SQL Statement	Mode of Table Lock	Lock Modes Permitted?				
		RS	RX	S	SRX	X
SELECT...FROM <i>table</i> ...	none	Y	Y	Y	Y	Y
INSERT INTO <i>table</i> ...	RX	Y	Y	N	N	N
UPDATE <i>table</i> ...	RX	Y*	Y*	N	N	N
DELETE FROM <i>table</i> ...	RX	Y*	Y*	N	N	N
SELECT ... FROM <i>table</i> FOR UPDATE OF ...	RS	Y*	Y*	Y*	Y*	N
LOCK TABLE <i>table</i> IN ROW SHARE MODE	RS	Y	Y	Y	Y	N
LOCK TABLE <i>table</i> IN ROW EXCLUSIVE MODE	RX	Y	Y	N	N	N
LOCK TABLE <i>table</i> IN SHARE MODE	S	Y	N	Y	N	N
LOCK TABLE <i>table</i> IN SHARE ROW EXCLUSIVE MODE	SRX	Y	N	N	N	N
LOCK TABLE <i>table</i> IN EXCLUSIVE MODE	X	N	N	N	N	N

Isolation Levels in Oracle

In Oracle Database, **Isolation Levels** determine how transaction integrity is visibly affected by the operations of other concurrent transactions.

READ COMMITTED	Default. A query sees only committed data.
SERIALIZABLE	Ensures full isolation, but can cause more blocking.
READ ONLY	For queries only; disallows DML.
READ COMMITTED	Default. A query sees only committed data.

READ ONLY

- ✓ Behavior:
- The transaction cannot issue DML (INSERT, UPDATE, DELETE).
 - Useful for reporting or long-running queries that must see a consistent state.

READ COMMITTED (Default)

✓ Behavior:

- Each query within a transaction sees only data committed before the query began.
- Other transactions' uncommitted changes are not visible.
- Successive queries in the same transaction may see different data if other transactions commit.

🔄 Allows:

- Non-repeatable reads: Data read twice might be different.
- Phantom reads: New rows may appear in subsequent queries.

✗ Prevents:

Dirty reads: Cannot read uncommitted changes from other transactions.

SERIALIZABLE

✓ Behavior:

- Ensures repeatable reads and consistency as of the start of the transaction.
- All queries within the transaction see a consistent snapshot.
- Oracle uses Multiversion Concurrency Control (MVCC), so it avoids locking rows for read.

✗ Prevents:

- Dirty reads: Cannot read uncommitted changes from other transactions.
- Non-repeatable reads Data read twice might be different –NO, it is **NOT** Allowed .
- Phantom reads (to an extent, though phantoms are controlled via predicate locks in other DBs)

⚠ Caveat:

Can lead to ORA-08177: can't serialize access for this transaction if data has changed since the snapshot.

Comparison of Isolation Levels in Oracle

Isolation Level	Dirty Reads	Non-Repeatable Reads	Phantom Reads	DML Allowed	Notes
READ COMMITTED	✗ No	✓ Yes	✓ Yes	✓ Yes	Default in Oracle
SERIALIZABLE	✗ No	✗ No	⊘ Mostly No	✓ Yes	May raise serialization errors
READ ONLY	✗ No	✗ No	⊘ Mostly No	✗ No	Used for consistent querying only

Isolation Levels in Oracle



Statements syntaxes – isolation levels:

SET TRANSACTION READ ONLY;

- This version defines the current transaction as read-only.

SET TRANSACTION READ WRITE;

- This version defines the current transaction as read-write and is the **default** setting.

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE | READ COMMITTED;

- When you specify **SERIALIZABLE**, a DML statement that attempts to modify a table already modified in an uncommitted transaction will fail. **SERIALIZABLE** transactions see only those changes that were committed at the time the transaction began, plus its own changes.
- If you specify **READ COMMITTED**, a DML statement that requires row-level locks held by another transaction will wait until those row locks are released. This is the **default**. Each query executed by a transaction sees only data that was committed before the query began.

Transaction -Best Practices

◆ **Best Practices:**

1. Keep transactions short and focused.
2. Always handle exceptions in PL/SQL to manage COMMIT/ROLLBACK properly.
3. Use SAVEPOINT judiciously in complex logic.
4. Avoid long-running transactions to prevent locks and contention.

✅ **Summary:**

Use READ COMMITTED for general-purpose transactions needing minimal blocking.

Use SERIALIZABLE if you need high consistency and are willing to trade off concurrency.

Use READ ONLY for reporting or data extraction when consistency is critical, and no updates are needed.

SESSION 5 :

Composite Data Types

- Create user-defined PL/SQL records
- Create a record with the `%ROWTYPE` attribute
- Create an `INDEX BY` table
- Create an `INDEX BY` table of records
- Describe the difference between records, tables, and tables of records



Composite Data Types



In PL/SQL, composite data types are types that can hold multiple values or elements—unlike scalar data types that hold only a single value. These are similar to data structures in other programming languages and are essential for managing collections or grouped data.

PL/SQL supports three main categories of composite data types:

Are of two types:

1. PL/SQL records -In PL/SQL, a record is a composite data type that allows you to group multiple fields (of potentially different data types) under a single variable. Think of it as similar to a "struct" in C/C++ or a "class without methods" in other languages.

1. PL/SQL collections

- INDEX BY tables or associative arrays
- Nested table
- VARRAY

Records

✓ Definition:

A record is a collection of fields (like a row of a table), each with its own name and data type.

◆ Types of Record Declarations:

- User-defined record type
- %ROWTYPE (based on a table or cursor)
- %TYPE (for field-level typing)

📝 Notes:

- Each record defined can have as many fields as necessary, but at least one.
- Fields without initial values are initialized to **NULL**.
- Can be defined in declarative part of any block, subprogram or package.
- You can declare and reference nested records. One record can be the component of another record.
- The **%ROWTYPE** attribute lets you declare a PL/SQL record that represents a row in a database table, without listing all the columns. Your code keeps working even after columns are added to the table.
- Records **do not need to be created in the database**.
- Useful in **procedures, functions, loops**, or when **returning multiple values**.
- Combine well with **cursors** and **bulk operations**.

✓ Syntax of a PL/SQL Record

1. Define a Custom Record Type

```
TYPE type_name IS RECORD (  
    field1 { datatype [NOT NULL] [:= | DEFAULT expr] | variable%TYPE |  
                table.column%TYPE | table%ROWTYPE }  
    [, field2, ...]);
```

2. Declare a Variable of That Record Type

```
rec_name1 type_name;  
rec_name2 table%ROWTYPE;
```

3. Assign Values and Access Fields

```
variable_name.field_name := value;
```

PL/SQL provides three types of collections:

1. **Associative Arrays** (Index-By Tables) -Key-value pair collection (like maps or dictionaries), indexed by number or string, only exists in memory.
 - Indexed by arbitrary numbers or strings.
 - No limit on number of elements.
2. **Nested Tables** - A Nested Table is a PL/SQL collection type that allows you to store an unordered set of elements. Unlike VARRAYs, nested tables can grow dynamically and can be sparse (i.e., have gaps in indexing due to deletions).Nested tables can be used in memory (PL/SQL) or stored in database tables.
 - Can be stored in a database column.
 - Can be sparse (have gaps in indexing).
3. **VARRAYs** (Variable-Size Arrays) -A VARRAY (Variable-size array) is a bounded, ordered collection of elements of the same data type. It has a maximum size specified at the time of declaration and is useful when the number of elements is known or relatively small.
 - Fixed-size array,
 - Index starts at 1,
 - Fixed maximum size.
 - Stored in-line with database rows (not sparse).

Syntax of Associative Arrays

1. Declaration Using Integer Index

```
TYPE array_type_name IS TABLE OF element_datatype  
INDEX BY BINARY_INTEGER;  
variable_name array_type_name;
```

2. Declaration Using String Index

```
TYPE array_type_name IS TABLE OF element_datatype  
INDEX BY VARCHAR2(size);
```

◆ Common Methods Used

Method	Description
EXISTS(i)	Returns TRUE if element at index i exists
DELETE(i)	Deletes the element at index i
DELETE	Deletes all elements
FIRST	Returns first index
LAST	Returns last index
NEXT(i)	Returns the next index after i
PRIOR(i)	Returns the previous index before i

✓ Key Characteristics

Feature	Value
Index Type	BINARY_INTEGER or VARCHAR2
Storage	Memory only (not in SQL)
Persistence	Not stored in database
Size Limit	Unlimited (until memory allows)
Sparsity	Supports gaps in index
SQL Usage	Not usable directly in SQL

Nested Table

◆ Syntax: Nested Table in PL/SQL Block

1. Define the Nested Table Type
TYPE nested_table_type IS TABLE OF element_datatype;
2. Declare a Variable of That Type
nested_table_var nested_table_type;

◆ Nested Table Methods

Method	Description
.EXTEND(n)	Adds n new elements
.DELETE(i)	Deletes element at index i
.DELETE	Deletes all elements
.EXISTS(i)	Returns TRUE if index i exists
.COUNT	Returns number of elements (non-null only)
.FIRST, .LAST	Returns first/last index

Operator	Description
MULTISET EXCEPT	The MULTISET EXCEPT operator is used to return the elements that are in one nested table but not in another . It compares element-by-element and is available only for nested table types , not associative arrays or VARRAYs.
MULTISET INTERSECT	The MULTISET INTERSECT operator is used to return the common elements (intersection) of two nested tables in PL/SQL. It compares elements of both nested tables and returns those that exist in both .
MULTISET UNION [ALL]	The MULTISET UNION operator is used to combine two nested tables into one collection that includes all elements from both , optionally keeping duplicates.

VARRAY Syntax in PL/SQL

◆ Basic Syntax (in PL/SQL block)

1. Declare a VARRAY Type

```
TYPE type_name IS VARRAY(max_size) OF element_datatype;
```

2. Declare a Variable of That Type

```
var_name type_name;
```

3. Initialize It (Optional)

```
var_name := type_name(value1, value2, ...);
```

⊘ Limitations

- You must specify a max size.
- Not suitable for large or highly dynamic datasets.
- Cannot delete specific elements (use TRIM to remove from end only).



Summary Table

Type	Indexed By	Max Size	Persistent (DB)	Can Be Sparse
Associative Array	BINARY_INTEGER or VARCHAR2	Unlimited	No	Yes
Nested Table	Integer	Unlimited	Yes	Yes
VARRAY	Integer	Fixed	Yes	No
Record	Field Names	Fixed fields	No (PL/SQL only unless in object types)	No

Detailed Comparison Table

Feature	Nested Table	VARRAY	Associative Array
Index Type	Integer (starts at 1)	Integer (starts at 1)	Integer or String
Bounded?	Unbounded	Bounded (max size defined)	Unbounded
Sparsity (Gaps Allowed)?	Yes	No	Yes
Persistence in SQL	Yes (can be stored in tables)	Yes (stored inline with rows)	No (PL/SQL only)
Suitable for SQL operations?	Yes	Yes	No
Extensible?	Yes (with EXTEND, DELETE)	Limited by defined size	Yes
Performance	Moderate (overhead for DB I/O)	Fast for small, fixed-size lists	Fast (in-memory only)
Index Flexibility	Positional only	Positional only	Positional or string-based
Common Use Case	Large, dynamic lists; DB storage	Small, fixed-size arrays	Temporary lookups, caches in PL/SQL
Feature	Nested Table	VARRAY	Associative Array
Index Type	Integer (starts at 1)	Integer (starts at 1)	Integer or String
Bounded?	Unbounded	Bounded (max size defined)	Unbounded

When to Use What?

Key Takeaways

- Use Associative Arrays for in-memory data processing.
- Use Nested Tables for dynamic, DB-stored collections.
- Use VARRAYs when the size is known and you prefer compact, inline DB storage.

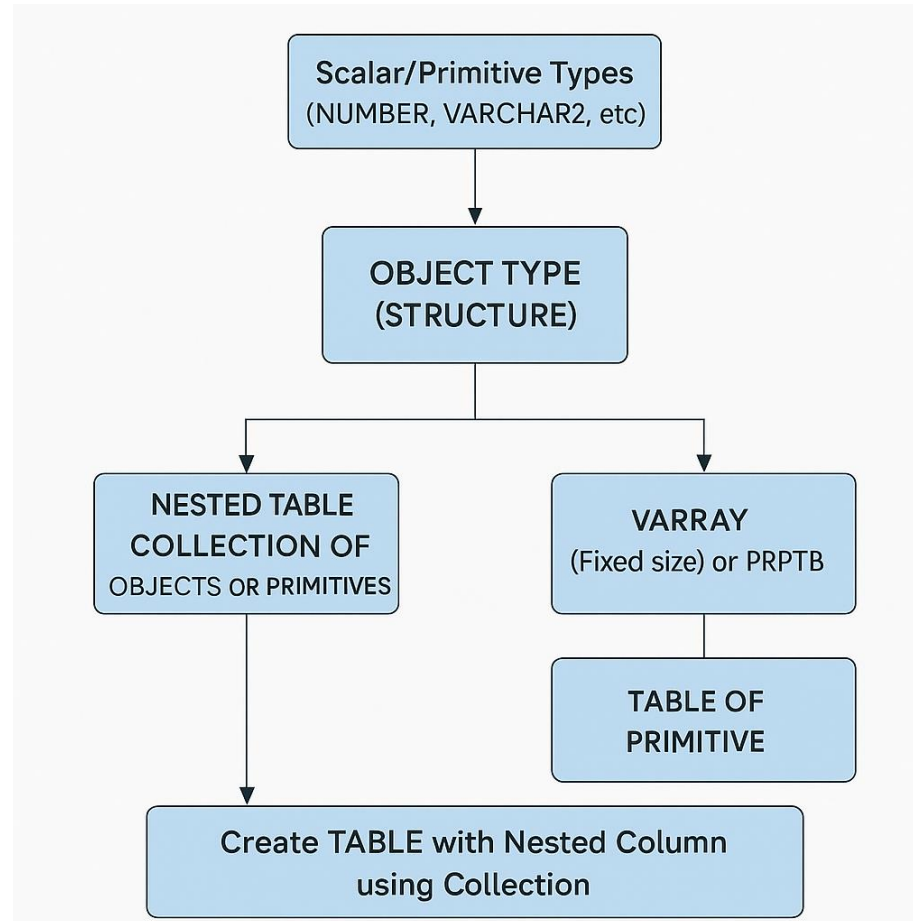
Use Case	Recommended Collection Type
You need to store collection in a DB column	Nested Table or VARRAY
You need fast, temporary, in-memory structure	Associative Array
You require sparse structure or deletions	Nested Table or Associative
You need fixed-size structure with compact I/O	VARRAY
You need string-indexed keys	Associative Array

User-Defined Types

Structured, reusable types like classes without methods (or with methods if needed).

Can be used in:

- PL/SQL variables
- SQL queries
- Table columns
- Nested tables



Summary & Homework



THEORY

[PL/SQL 101 Part 8](#)

PRACTICE

- [Object_Collection_Comparisons_Part1](#)
- [Introduction to Collections](#)

HOMEWORK

- [Quiz on Fetching a Single Row](#)
- [Quiz on Bulk Processing with FORALL and BULK COLLECT](#)